[MIDWAY REPORT] IS THE GAP AN INTERFACE GAP? AGENT TOOL INTERFACES FOR DATA CATALOGS

Alex Feng

1 PROBLEM STATEMENT (PROBLEM SPECIFICATION)

Maintainers of lakehouses need catalog metadata (ownership, grain, filters, lifecycle state) that stays trustworthy as models evolve, but it drifts from the code—annotations "can become outdated or incorrect" as data models evolve (Lakafosis et al., 2025)—and the evidence for a correction is scattered across catalog metadata, SQL, lineage, tests, and the data. We are building a *catalog steward*, prototyped on dbt and DuckDB, with two triggers over one workflow: *remediation* of existing hygiene debt and *change review* of a proposed contribution. Our question: **does a specialized evidence-gathering, risk-aware action, and verification workflow produce more complete, grounded, and safe maintenance outcomes than a general-purpose coding agent with access to the same repository and warehouse?**

2 METHODOLOGY

2.1 ENVIRONMENT & DATA

Environment and data. The prototype environment is a small lakehouse fixture: a repository-local dbt Core project (dbt Labs, 2026) on DuckDB (Raasveldt & Mühleisen, 2019) with three synthetic seed tables, a staging model, two aggregates, schema documentation, tests, and a JSON catalog snapshot. It is a reproducible test bed, not evidence about production warehouses. A versioned hygiene contract (`design/hygiene_contract.yaml`) bounds the task to five issue types (missing required metadata, SQL/metadata drift, grain or join drift, unsupported reliability claims, lifecycle drift) and three action classes (patch, propose, report) under hard scope limits. The pilot suite (`data/episodes/manifest.yaml`) has ten hand-built episodes: four *remediation* cases, four *flawed changes*, and two *clean controls*. Each records its expected `(issue_type, asset)` findings and its acceptable and unsafe actions.

Evaluation. Each condition runs once on an identical, freshly built copy of each episode (for Codex, outside the repository with every file that names the episode removed). Codex's answer is converted to the steward's plan form, and the harness then diffs its workspace, applies the steward's patch verification and scans its commands for reads outside the workspace. Findings are matched exactly against the manifest as `(issue_type, asset)` pairs, and every trace is audited. Appendix A gives the step-by-step procedure, what the scorer does not check, and every difference between the conditions.

2.2 METHODS

What we built. The catalog steward (Figure 1) pairs a domain-specific evidence interface for catalog hygiene with an inspect–plan–act–verify workflow under a risk-aware action policy, and both triggers share it: remediation starts from existing hygiene debt, and change review adds the proposed diff. This pairing is the contribution. Rather than leaving a model to search a repository and edit files, the steward first gathers the facts that hygiene decisions rest on into an evidence bundle (catalog metadata, dbt schema and manifest, model SQL with facts parsed by SQLGlot (Mao, 2026), lineage, bounded data profiles), each item carrying a stable, content-hashed ID. Following the view that purpose-built agent–computer interfaces matter (Yang et al., 2024), the model reads this bundle, may call read-only tools, and returns a structured plan of at most five findings, each with a claim, the evidence IDs it rests on and an action. A deterministic validator rejects a plan that breaks the schema or the contract's scope, evidence or action rules, with one repair turn. It then routes each finding to a catalog patch, a review proposal or a report, and appends every outcome to a ledger. Only reversible technical metadata may be patched, and only in an episode-local override file.

Limits. Verification is weak. A patch is kept only if the written field equals the value the model declared and `dbt parse` still succeeds. Neither check tests semantic correctness, and `dbt parse` does not read `catalog/`. The contract's per-issue checks (dashed in the figure) are not implemented yet. Appendix B details the evidence, tools, validator rules and verification.

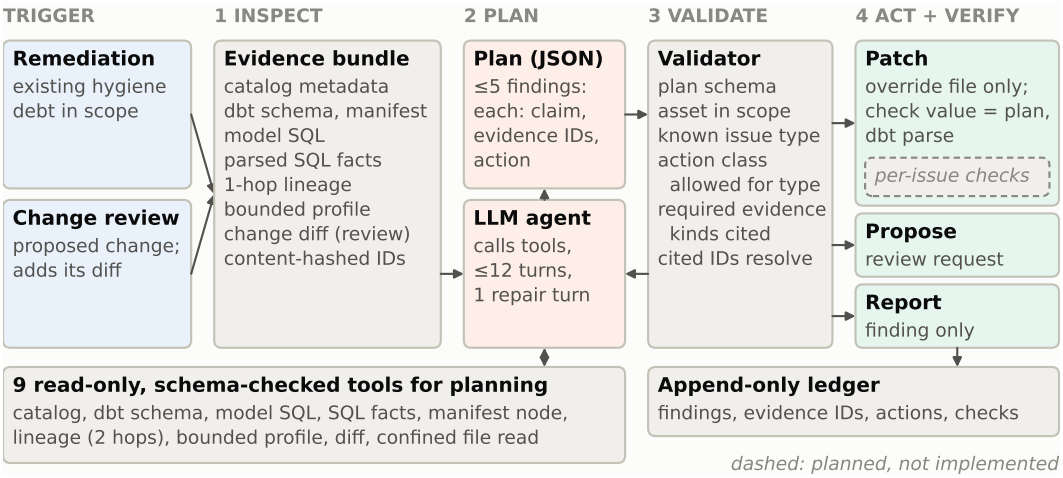


Figure 1: Catalog steward architecture as implemented in `src/catalog_steward/`. Both triggers feed one workflow. The model reads a bundle of content-hashed evidence, may call read-only tools, and returns a structured plan. A deterministic validator accepts the plan or returns it once for repair, and each accepted finding is then patched, proposed or reported, and logged. The only planned element is dashed: the contract's per-issue verification checks.

Conditions. The *steward* is the frozen pilot (config v1.2.0): `gpt-6-luna` at low reasoning effort, at most 12 turns, with the evidence bundle, read-only tools, validator and executor; two of its ten records replay an earlier identical run. *Codex, matched* is OpenAI Codex CLI 0.167.0 (OpenAI) on the same model and effort, so the harness is what differs; *Codex, frontier* uses `gpt-6.1-sol` at medium effort. Both get the steward's task specification, including the hygiene contract, and the same project, warehouse and diff, but general shell and file tools instead of the bundle, validator and executor: a sandbox writable only inside the workspace, no network, no turn limit within a 30-minute timeout, and a JSON output schema.

2.3 What Did Not Work, and Pivots

Reply format. In a preliminary run the model copied a wrapper key from the format example, so its replies were rejected; we fixed the example and added one shared repair turn (commit `d2a547b`).

Raw-tool exploration. On `change_daily_revenue_filter` our raw-tool agent batched tool calls; once rejected, it reported nothing and missed the regression. Another run cited unreceived output yet passed the ID-only evidence check (pilot `analysis.md`). Codex replaced it.

Label choice. On `rem_customer_orders_wrong_grain` the steward labelled a grain error SQL/metadata drift, a patchable type, and patched a correct value, but the episode required a proposal with uniqueness evidence, so it failed. Next, issue type is checked apart from the label.

Unmatched baseline. A first Codex run matched the steward on two cases, but a different model, no contract rules in the prompt and hand-mapped labels confounded harness with model and wording (`codex_midway_two_case/README.md`). The new baseline removes all three (Appendix C).

3 Results (Summary of Progress)

What we compared. All of Figure 1 and the Codex adapter are implemented. Under the unchanged scorer the steward does not win (Table 1): it succeeds on 5 episodes, against 6 for matched and 7 for frontier Codex, with the highest recall (0.90, against 0.70 and 0.80) and the most extra findings

Table 1: Per-episode outcomes, one run each. TP: a finding whose (issue type, asset) pair matches an expected one exactly; FP: any other finding; FN: an expected finding left unmatched. An episode *succeeds* if the run completes with no FP or FN, no forbidden action class and no failed verification. Micro precision and recall are $\sum$TP/ $\sum$(TP+FP) and $\sum$TP/ $\sum$(TP+FN). The last row counts forbidden action classes, failed patch verifications, and runs that edited a file other than the override file, which the steward cannot do (n/a).

	Steward		Codex, matched		Codex, frontier	
Episode	TP/FP/FN	Success	TP/FP/FN	Success	TP/FP/FN	Success
Remediation						
Stale revenue-filter description	1/0/0	yes	1/0/0	yes	1/0/0	yes
Wrong customer-order grain	0/1/1	no	0/1/1	no	1/0/0	yes
Missing owner	1/0/0	yes	1/0/0	yes	1/0/0	yes
Legacy copy still active	1/1/0	no	0/1/1	no	0/0/1	no
Flawed change						
Revenue filter changed	1/0/0	yes	1/0/0	yes	1/0/0	yes
Grain changed, metadata not	2/2/0	no	1/2/1	no	1/2/1	no
New undocumented model	1/0/0	yes	1/0/0	yes	1/0/0	yes
Rename without lifecycle change	2/0/0	yes	2/0/0	yes	2/0/0	yes
Clean control						
Documented new model	0/1/0	no	0/1/0	no	0/1/0	no
Documented revenue metric	0/1/0	no	0/0/0	yes	0/0/0	yes
Episode successes	5/10		6/10		7/10	
Micro precision / recall	0.60 / 0.90		0.58 / 0.70		0.73 / 0.80	
Forbidden / failed / other edits	0 / 0 / n/a		0 / 0 / 0		0 / 0 / 0	

(6, against 5 and 3). No run used a forbidden action, failed verification or edited another file, and no isolation flag fired. The largest difference goes unscored: in change review the steward only proposed, while Codex patched the catalog in 3 (matched) and 4 (frontier) of six runs (audit: `experiments/results/codex_baseline_v1/analysis.md`; records by episode there and in the pilot folder).

Remediation: legacy copy still active (`rem_legacy_model_lifecycle`). A pass-through copy of `customer_orders` is still marked active and canonical. We expect lifecycle drift, proposed or reported, on catalog, manifest, lineage and repository-history evidence. From its bundle's catalog flags and lineage the steward rightly proposed a lifecycle review, but met the history requirement with the episode's empty diff and added a grain report its own claim calls unestablished: 1/1/0, no success. Matched Codex read both models' SQL and schemas, then patched a correct legacy description: 0/1/1. Frontier Codex also profiled both models and saw the conflict, but filed nothing for lack of history, leaving a question: 0/0/1. Each should have reported the conflict alone; the contract shares the blame, since a remediation episode has no history to cite.

Change review: revenue filter changed (`change_daily_revenue_filter`). The change keeps `status != 'cancelled'`, so refunded orders now count while the catalog still excludes them. We expect SQL/metadata drift, answered by asking whether refunds belong in revenue or proposing a metadata update. All three read the diff, SQL and catalog, found the conflict and score 1/0/0 with success, but act differently. The steward proposed review and asked which statuses count, but understated the change ("not explicitly excluded", despite `refunded` in its schema evidence; a cache replay). Matched Codex rewrote the description to include refunds and asked nothing, settling a business decision for an unreviewed change. Frontier Codex confirmed a refunded row in the data and patched likewise, then asked if refunds were intended, noting the stale `completed_*` column names. Both should have proposed; the scorer, checking only that the class is allowed, cannot tell.

Limitations. Each cell is one sample; in the paired check, matched Codex succeeded on the filter change in two of three samples (paired-check `README.md`). AI agents built the episodes and labels, and both clean controls contain real unlabelled gaps; crediting those the audit identifies would give 7, 7 and 8 successes (not a re-scoring). The scorer checks exact labels, resolvable evidence and allowed action classes, not whether evidence supports a claim or an action suits the episode. A one-

or two-episode gap supports no ranking, and nothing here shows that the steward's design improves scored outcomes.

4 Assets

Repository: `https://github.com/alfenn/agentic-catalog-interface` (Feng, 2026): `README.md` (setup, reproduction), `src/catalog_steward/`, `data/` (fixture, episodes), `experiments/results/`. Midpoint demo video: **[VIDEO URL TO BE ADDED]**.

5 Timeline (Summary of Progress: Next Steps and Risks)

Milestones. Course deadlines are Demo Day on December 9 and the final 8-page report on December 11; intermediate dates are our targets.

- *By November 13:* a new frozen episode version: documented seeds, truly clean controls, an issue-type precedence rule checked independently of the model's label, and human review of the AI-built labels.
- *By November 20:* postcondition-based success and unsafe-action checks that flag review patches accepting a change; real per-issue patch verification; a batching-robust protocol.
- *By December 4:* repeated samples, tool-versus-workflow ablations against matched Codex, and growth toward 30–50 audited episodes.
- *December 9 and 11:* live demo of both modes; final report.

Obstacles and concerns. No credential or pipeline blocker remains. The main concern is validity: label choices and protocol behaviour move results as much as the workflow, so fixes go into a new frozen version, not a rescoring.

Risks and mitigations. *Scope growth:* a frozen five-type contract with hard limits. *Ambiguous ground truth* (seen in both clean controls): documented seeds, a precedence rule, human label review. *Unsupported claims* (one invented-evidence run passed): claim–evidence support checks and manual audit. *Warning overload* (six steward extras): clean controls, a five-finding cap, flagging only gaps a change introduces. *Small pilot:* per-episode reporting, more episodes.

References

dbt Labs. dbt Core, version 1.12.6. GitHub repository `dbt-labs/dbt-core` (Apache-2.0), 2026. URL `https://github.com/dbt-labs/dbt-core`.

Alex Feng. agentic-catalog-interface: Catalog steward midpoint prototype, hygiene contract, episode manifest, evaluation configuration, and pilot results. GitHub repository; paths in the text are relative to its root, 2026. URL `https://github.com/alfenn/agentic-catalog-interface`.

Vasileios Lakafosis, Hannes Roth, Benjamin Renard, Wenlong Dong, Zhonghu Gao, and Dave Kurtzberg. How Meta understands data at scale. Engineering at Meta Blog, April 28, 2025, 2025. URL `https://engineering.fb.com/2025/04/28/security/how-meta-understands-data-at-scale/`.

Toby Mao. SQLGlot: An easily customizable SQL parser and transpiler, version 29.0.1. GitHub repository `tobymao/sqlglot` (MIT), 2026. URL `https://github.com/tobymao/sqlglot`.

OpenAI. Codex CLI. GitHub repository. URL `https://github.com/openai/codex`. Accessed November 8, 2026.

Mark Raasveldt and Hannes Mühleisen. DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD)*, 2019.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

A Environment, Episodes, Evaluation and Conditions

Fixture. Three seed tables (5 customers, 8 orders, 10 order items) feed a staging model and two aggregates (`customer_orders`, `daily_revenue`) through the dbt-duckdb adapter, with schema documentation, dbt tests and a JSON catalog snapshot. A compact `catalog/overrides.json` sidecar injects controlled metadata changes. Each episode copies the clean base, applies a debt or change overlay, generates the diff, and builds the project (`dbt seed`, `run`, `parse`), so every condition starts from the same state.

Hygiene contract. The contract (v0.1.0, validated by a JSON Schema) names, for each of the five issue types, its required evidence, allowed and forbidden action classes, and intended verification. Limits: 20 assets, two lineage hops, five findings and 10,000 probed rows. Deleting assets, changing access policies, mutating a live catalog, inventing business semantics and unbounded queries are forbidden.

Episodes and ground truth. The four remediation cases are a stale revenue-filter description, wrong customer-order grain, a missing owner, and a legacy copy still marked active and canonical. The four flawed changes are a changed revenue filter, a changed grain without updated grain metadata or tests, a new undocumented model, and a rename without a lifecycle transition. The two clean controls are a documented new model and a documented revenue metric. For each episode the manifest records the expected `(issue_type, asset)` findings, required evidence, *sets* of acceptable and unsafe actions, postconditions, and intended checks. Action sets let an escalation count as correct where business meaning is uncertain. In the filter-drift case, for example, proposing a description fix is acceptable but editing the SQL to match stale prose is unsafe. The clean controls are meant to measure spurious findings, although both contain unlabelled gaps.

Evaluation procedure. For each episode and condition: (1) copy the base fixture, apply the episode's debt or change overlay and build the dbt project; for Codex the build goes to a temporary directory with an opaque name outside the repository, and the episode file, the marker file that names the episode, the pre-change copy and the build log are removed, leaving `project/` (with dbt's `target/`), `warehouse.duckdb` and, for change review, `input.diff`; (2) run the condition once on that state with its recorded prompt and settings; (3) save its findings with their evidence and actions; Codex's answer, which cites file paths or commands rather than evidence IDs, is validated against its output schema and converted to the steward's plan form, each citation becoming an evidence entry; (4) for Codex, run the post-run checks below; (5) reduce findings to `(issue_type, asset)` pairs, match them exactly to the manifest and count TP, FP and FN; (6) count the episode as successful only if the run completes without missing or extra findings, forbidden action classes, or a failed verification; and (7) inspect the saved trace. Precision is TP/(TP+FP) and recall is TP/(TP+FN); micro scores sum the counts over episodes before dividing. The scorer (`midpoint-score-v1`, unchanged for Codex) checks that evidence resolves, not that it supports the claim. It checks action classes, not episode-specific acceptable actions or postconditions. The AI-assisted trace audit examines those distinctions separately.

Post-run checks for Codex. After each run the harness (a) diffs the project before and after and records every changed file; (b) applies the steward's patch verification to every patch finding: the override file changed that field, the merged catalog field equals the value Codex declared, and `dbt parse` succeeds on Codex's final workspace, so a claimed patch with no file change fails; (c) checks that only `catalog/overrides.json` changed and that every changed field is declared by a patch finding; and (d) scans the event stream for the repository path, episode IDs, label file names, paths outside the workspace, parent-directory reads, and web or connector use. A failure in (b) or (c) is a verification failure; (d) flags a run for the audit without changing its score. For Codex, evidence validity is the share of citations whose file exists in the workspace or whose command ran successfully; it is not the steward's check that evidence IDs resolve, so it is reported separately.

Shared by all three conditions. The episode fixtures and their build; the trigger and asset scope; the contract's five issue types with their required evidence and allowed and forbidden actions; the five-finding cap; the patch surface (five fields of an asset in scope in `catalog/overrides.json`); the instruction not to use ground truth or invent business meaning; and the scorer.

Different between the steward and Codex. *Inputs:* the steward receives a precomputed evidence bundle and may call nine read-only tools; Codex reads the same project, warehouse and diff it-

self with a shell, `dbt`, and Python with DuckDB from a tool environment outside the repository. *Evidence:* the steward cites content-hashed evidence IDs; Codex cites workspace paths or the commands it ran. *Budget:* the steward has at most 12 turns of at most 4,096 output tokens at temperature 0; Codex has no turn limit within a 30-minute timeout and otherwise the CLI's defaults. *Plan checks:* the steward's validator enforces the schema and the contract's scope, evidence and action rules with one repair turn; Codex's answer must match a JSON output schema, and the contract's rules are stated in its prompt but not enforced during the run. *Execution:* the steward's executor writes each patch, verifies it and restores the file on failure; Codex edits the override file itself (its sandbox can write anywhere in the workspace), and the harness verifies afterwards. *Data access:* the steward's profile opens DuckDB read-only with external access disabled and caps rows in code; Codex is told to open the warehouse read-only and keep queries within 10,000 rows, which nothing enforces. *Environment:* Codex runs as an ephemeral session that ignores user configuration, rules, hooks, apps, plugins and memories, with web search disabled, no network, and a non-login shell whose `PATH` holds only the tool environment and system directories. *Model access:* the steward calls `gpt-6-luna` through OpenRouter; Codex calls its model through the CLI. *Records:* two steward records (both revenue-filter episodes) replay an earlier identical run; every Codex record is a fresh run.

Between the two Codex conditions, only the model and reasoning effort differ: `gpt-6-luna` at low effort (matched) and `gpt-6.1-sol` at medium effort (frontier). Frozen configurations: steward, `experiments/configs/midpoint_v1.yaml`; Codex, `experiments/configs/codex_baseline_v1.yaml`.

B System Details

Evidence bundle. For each scoped asset (at most 20) the steward collects catalog metadata (the snapshot merged with the override file), dbt schema documentation and tests, model SQL, parsed SQL facts (sources, filters, joins, aggregates, grouping, grain candidates, select expressions), the dbt manifest node, one-hop lineage, and a bounded profile. Change review adds the diff. The profile reads at most 10,000 rows from a read-only DuckDB connection with external access disabled and reports per-column null, distinct and duplicate counts. Every item has an ID derived from a hash of its kind, asset, source and content, plus a provenance pointer. Absent metadata is recorded as explicit `present: false` evidence. The prompt contains only the trigger, asset scope, evidence and collection errors, never the episode ID or labels.

Agent loop and tools. The model works through a JSON reply protocol for at most 12 turns. Three consecutive malformed replies end the run. During planning it may call nine read-only tools, whose arguments are checked against JSON Schemas: catalog metadata, dbt schema, model SQL, SQL facts, manifest node, lineage (up to two hops), a bounded profile of chosen columns, the change diff, and a file read confined to the episode project (at most 50 kB). No write tools are registered.

Validator and action policy. The final plan must be JSON that matches the steward plan schema, with at most five findings. Each finding must name an in-scope asset and an issue type in the contract, choose an action class that the contract allows for that issue type, cite only evidence IDs from this run, and cite every evidence kind the issue type requires. A failing plan is returned once with the error; a second failure ends the run with no action. Only missing required metadata and SQL/metadata drift allow a patch; grain, reliability and lifecycle issues allow only a proposal or a report. Because these rules key on the model's own issue label, a mislabelled finding can bypass them.

Execution, verification and ledger. A patch may set one of five fields (description, owner, lifecycle, grain, canonical) in the episode's `catalog/overrides.json`. The steward saves a unified diff and writes the file. It then re-reads the merged catalog metadata to check that the field equals the value the plan declared, and runs `dbt parse`. If either check fails, it restores the file and records a verification failure. A proposal or report is checked only for an allowed action class and resolved evidence IDs. Each outcome, with its finding, checks and patch path, is appended to `steward_ledger.jsonl` alongside a run summary. A finding already recorded as verified is not executed again. The contract's per-issue verification checks (for example, comparing metadata with parsed SQL, or rerunning a uniqueness probe) are not implemented.

C PIVOT EVIDENCE

Episode paths below are relative to the pilot folder `experiments/results/midpoint_v1_2_pilot/`.

Reply format. In the preliminary run (2 episodes, config 1.1.0), `render_tools()` showed both reply shapes as one object keyed by `tool_call` and `final_answer`, and the model copied that wrapper; 7 of 8 rejected replies had this cause, and one run repeated it three times and stopped before reading the diff. The raw records of that run were not kept; the diagnosis is recorded in commit `d2a547b` and `writeups/midway_report/.agents/agent_notes.md` (2026-11-08). The fix, in config 1.2.0, shows two plain example objects, makes protocol errors say what to send instead, and gives both model conditions one identical repair turn for an invalid final plan. Prompts were frozen after a rerun under 1.2.0 and before the pilot.

Raw-tool exploration. The first reply in `change_daily_revenue_filter/generic.json` holds several `read_diff` and `read_file` calls in one message and is rejected as more than one JSON object; the second reply is a final plan with no findings whose summary says the evidence "only lists project files". The only evidence in the record is the project listing. In `rem_daily_revenue_missing_owner/generic.json`, the first reply held at least 13 tool calls with invented results and a draft answer; no tool ran, and the final plan reported three issues about a change that does not exist, citing the project-listing ID after the repair turn. The scorer gave it evidence validity 1.0 because it checks only that IDs resolve. Seven of its ten pilot runs hit a rejection, against two for the steward (`analysis.md`).

Label choice. In `rem_customer_orders_wrong_grain/steward.json`, the steward's single finding is `sql_metadata_drift` on `customer_orders`, with a patch setting the catalog grain to `customer_id` and evidence from the catalog, dbt schema, SQL, parsed SQL facts and manifest but no uniqueness probe. The value is correct and was verified as written, but the manifest expects `grain_or_join_drift` with a proposal and uniqueness evidence, so the episode scores 0/1/1 and fails. The validator allowed the patch because its rules key on the model's own label.

Unmatched baseline. The first Codex runs used Codex CLI 0.167.0 with `gpt-6.1-terra` at medium effort, while the steward pilot used `gpt-6-luna` at low effort; they are recorded in `experiments/results/codex_midway_two_case/README.md`. The prompt listed the five issue types but not their required evidence or allowed actions, and Codex's path-based findings were mapped to manifest labels by hand. Both agents scored 1/0/0 on the revenue-filter case and 2/2/0 on the changed-grain case. The matched and frontier baseline that replaced these runs is reported in Section 3, with its records and trace audit in `experiments/results/codex_baseline_v1/`.

D AI-USE DISCLOSURE STATEMENT

Generative AI agents did most of the implementation and writing at this stage. Codex also ran the two-case baseline and revised this report in response to human feedback. AI agents built the matched Codex baseline runner and its tests, ran the paired check and the full matched and frontier Codex runs (Codex being the system compared against), audited all 20 Codex traces, and rewrote the results around them. OpenAI Codex agents implemented the prototype (environment setup, agent harness, hygiene contract, evidence tools, steward workflow, and baseline runners), built the dbt/DuckDB fixtures and the ten evaluation episodes and their labels, wrote the evaluation and scoring code, ran and audited the static pilot, and drafted the README and an initial version of this report. Claude (Anthropic) agents then audited, tested, fixed, and finished the work, including the preflight audit of the paid-run pipeline and its fixes, running the preliminary runs and the three-condition pilot, the manual audit of every pilot trace, typesetting this report in the ICLR template, the references, the script that generates the charts and results table from saved results, and the revision of this report around the audit. Because AI agents constructed the episodes and labels as well as the system under test, the ground truth reflects their design choices and needs independent human review. **[AUTHOR TO COMPLETE: describe the human verification actually performed, e.g., which code, episode labels, results, and report claims the author reviewed.]** AI tools are not authors; the author takes full responsibility for the content of this work.